PRISM | Tech Management, SRI-B

CLASH OF THE **CLAWS**

RVCE_calico
Path Pilot: Your AI-Powered Daily Commute Assistant

Aditi Shastri, Aditya KGU, Angela Varghese

Path Pilot

Problem Statement

The Problem: Analysis Paralysis in Daily Commute

What? Overwhelming choices, traffic unpredictability, the stress of being on time.

Who? Everyday commuters in Bengaluru (and beyond, other cities and towns).

Why? 84% of female and 70% of male employees in Bengaluru's corporate sector report they "always or often" feel stressed during their daily commute[1].

Theme: Daily Utility and Productivity
Why did we choose this theme?

Pain Point: Commuters waste time and mentally energy deciding how to most efficiently get to their destination without burning a hole in their wallet.

Goal: Eliminate the stress of planning, to focus on the things that matter.

How it works: A Day in the Life of a Happy Commuter

Input: Your calendar (e.g., “Meeting at 3 PM in Indiranagar).
Analysis: Real-time traffic + transit data (metro timings, bus routes).
Output: “Leave by 2:15 PM, Take metro (Line 1) to avoid traffic.
ETA: 2:50 PM
Reminder: Telegram alert 30 mins before

Who Benefits?
Urban commuters (Bengaluru, Mumbai, Delhi, etc.)
Busy professionals juggling multiple appointments.
Anyone who wnats to outsource the thinking to an AI.

[1] Aravindakshan, A., & Vijayan, K. (2026). *“Traffic congestion’s differential impact on stress and productivity: A gender-specific analysis of Bangalore corporate employees.”* The International Journal of Indian Psychology, 14(1).

Idea

Current Solution & Gaps

Existing Software
Google Maps /Waze

Public Transit Apps: NammaBMTC, NammaMetro

Calendar Apps: Google Calendar

Ride-Hailing Apps like Uber, Ola, Rapid etc.

No Proactive Decision-Making: Users still manually decide how to react to changes (e.g., "Should I leave now or wait?").

Hyper-Local Gaps: City-specific challenges (e.g., unreliable bus timings, mixed traffic modes) are not addressed by global tools.

Lack of unified platform that removes core bottle neck of making efficient decisions with cost-effectiveness.

Opportunity

- Unified AI Orchestrator - A single platform that connects calendars, traffic, transit, and messaging to automate daily planning.
- Dynamic Adaptability - Real-time adjustments (e.g., "Metro delayed? Switch to a bus + auto combo").
- Context-aware reminders (e.g., "Leave now—traffic is worse than usual").
- Hyper-Local Optimization:
- Tailored for Bengaluru’s chaos (e.g., prioritizing metro over roads during peak hours).
- Move from "What should I do?" to "The AI handled it."—reducing cognitive load and stress.

Idea

Your Solutions

Core Ideas

- Authenticated Workspace – Users sign in, maintain persistent chat history, and connect external services securely
- Multi-Model Flexibility – Route requests across OpenAI, Anthropic, Google, Groq, and OpenRouter for optimal performance
- Voice-First Design – Talk to your assistant via browser microphone with speech synthesis output
- Database-Aware Tools – Safe read queries with write operation previews and confirmation flows
- Integrated Services – Google Calendar, Telegram, commute planning, and more work seamlessly within the chat

Key Features

- ✅ Telegram Integration

Bot token setup and webhook support
Outbound notifications sent directly to Telegram

- ✅ Voice Assistant: Browser speech input (microphone)

Multiple output options: browser speech synthesis, ElevenLabs, or server-side transcription

- ✅ Calendar & Commute Planning

Google Calendar OAuth integration
TomTom routing for route planning and traffic estimates
Context-aware assistant recommendations
Excel export for tabular data

- ✅ Multi-Provider AI Routing

Automatic model selection and fallbacks
Vendor-agnostic architecture

AI SDK used: Vercel AI SDK
It is an application-layer SDK for building AI-powered web apps.
Vercel AI SDK was chosen because it provides production-ready streaming, tool calling, multi-model support, and tight integration with Next.js and React, which fits a real-time AI assistant platform better than OpenClaw, which is more suited for autonomous/self-hosted agent experimentation,

Tech stack:

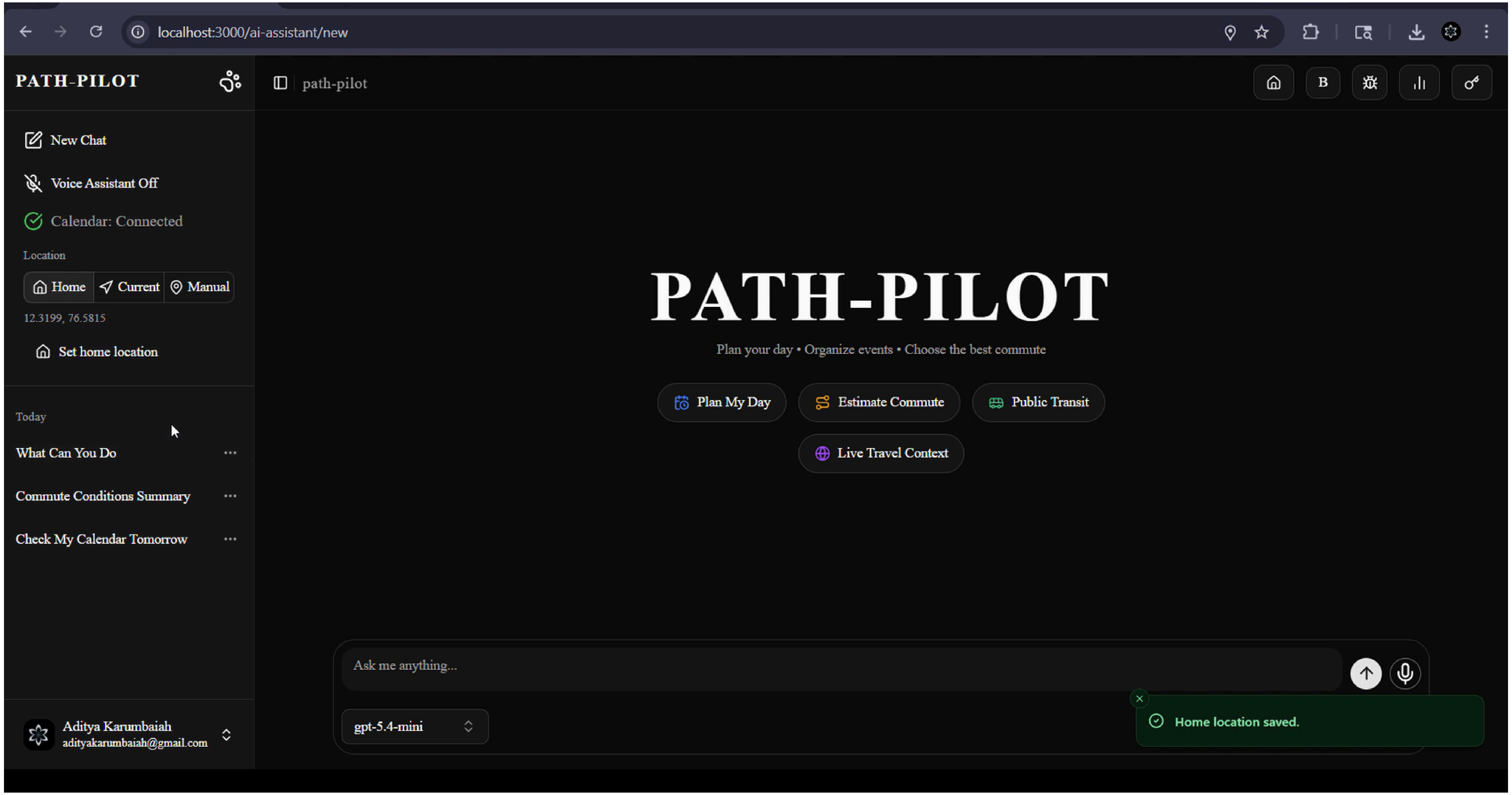
- Framework: Next.js 16 (App Router)
- Language: TypeScript
- UI: React 19 + Tailwind CSS
- AI: Vercel AI SDK
- Backend: Supabase Auth & Database APIs
- Integrations: OpenAI, ElevenLabs, Google Calendar, Telegram Bot API, TomTom

Idea

Demo/Product Walkthrough

We begin at the very beginning, the landing page

Demo Video available in GitHub repository and at the end of the presentation



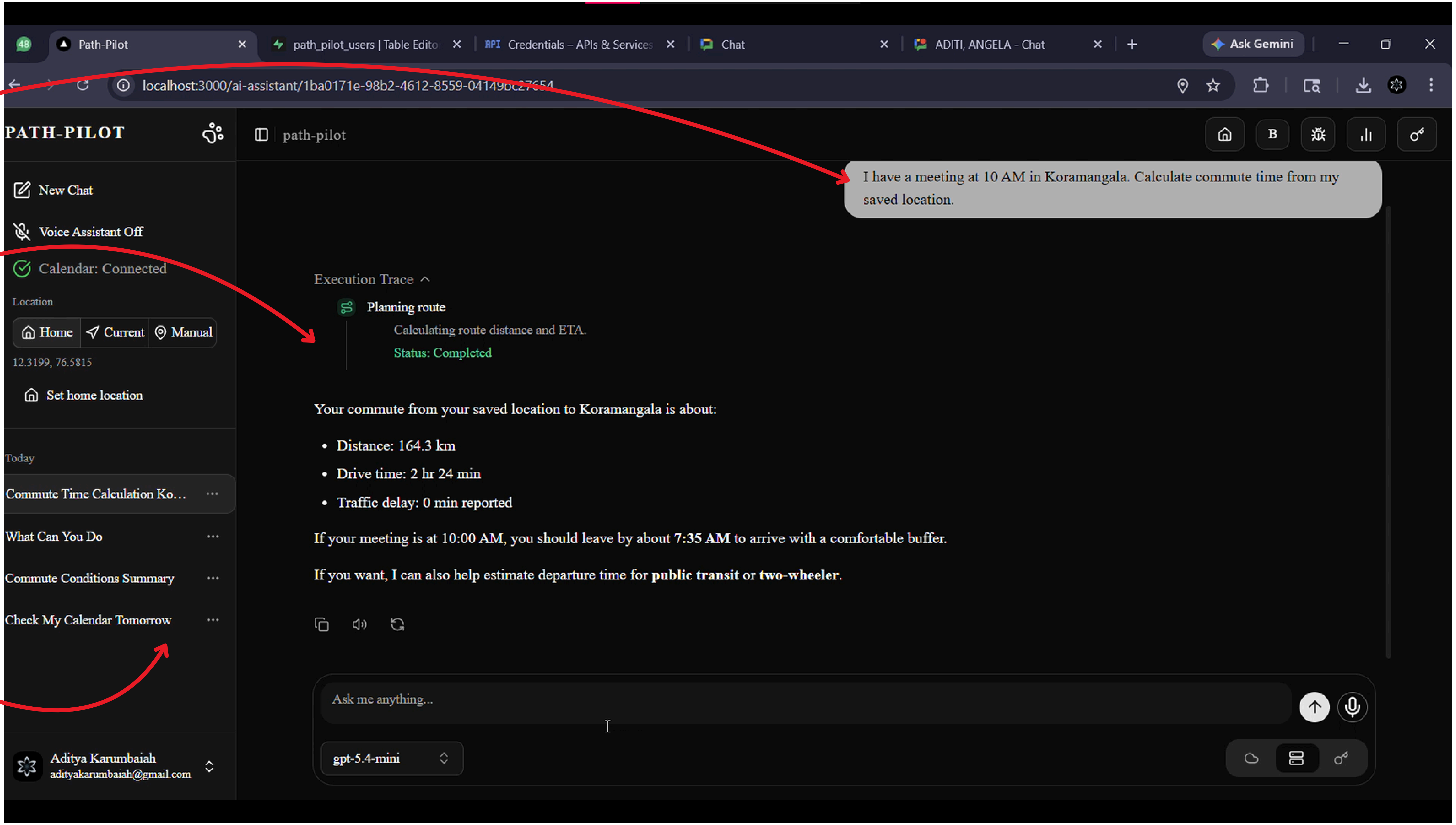
Idea

Demo/Product Walkthrough

The user enters information related to planning or setting constraints, in this case, a meeting at 10 AM in another part of town.

Route is planned

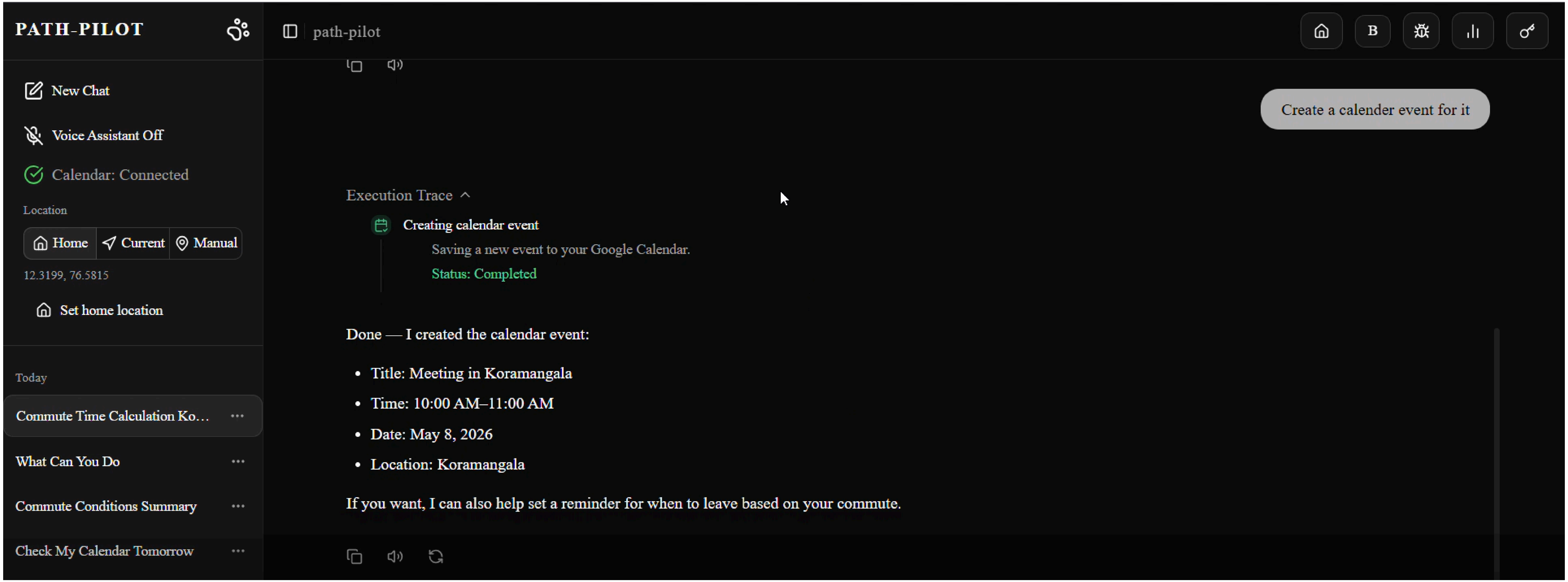
Chat history shows various questions related to functionality and usability.



Idea

Demo/Product Walkthrough

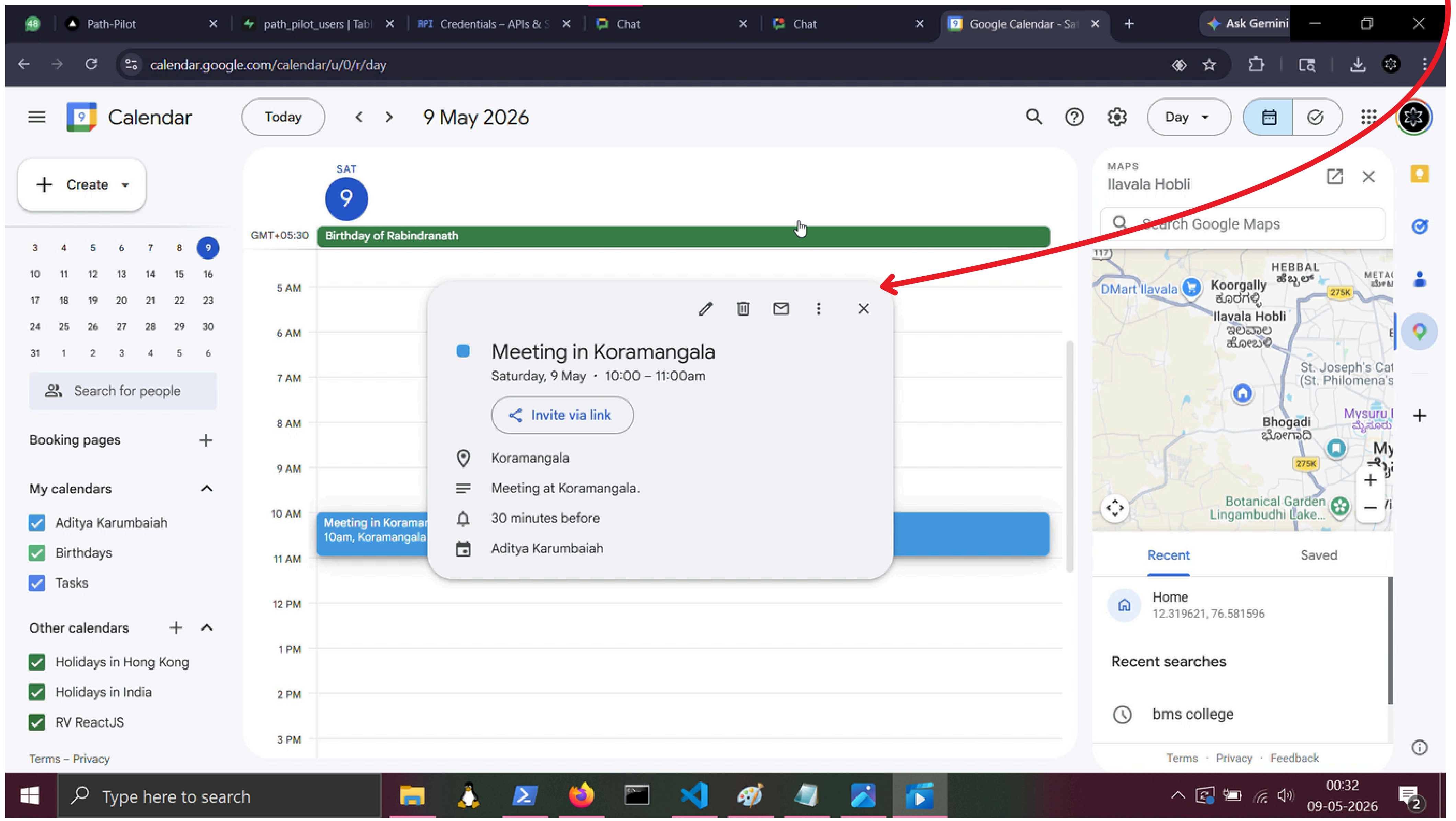
User can create new calendar events with natural language prompt.



Idea

Demo/Product Walkthrough

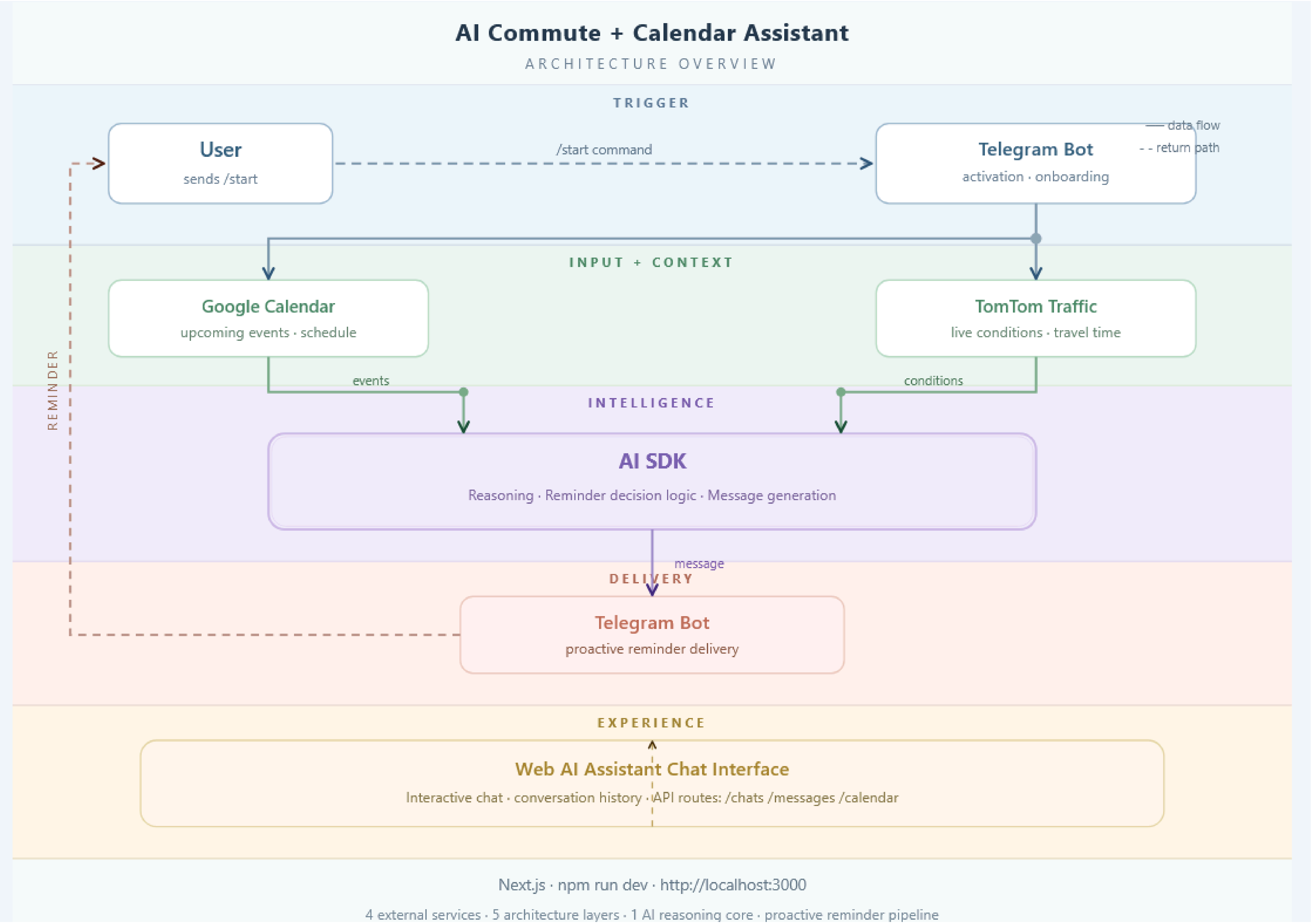
Calendar event created with the help of Google Calendar API, path is also routed from source to destination



Telegram Reminders sent 30 mins before actual start time.

Idea

Tech & Architecture



Idea

Tech & Architecture

TECH STACK

Service / Tool	Role in System	Layer	Integration Type
Google Calendar API	Source of user schedule — fetches upcoming events for reminder timing	Input	OAuth 2.0 · REST API
TomTom Traffic API	Real-time traffic conditions; adjusts reminder lead time dynamically	Context	REST API · API Key
Telegram Bot API	User onboarding via `/start` and outbound reminder delivery	Trigger + Delivery	Webhook · Bot API
AI SDK (Claude / Anthropic)	Core reasoning engine — decides timing, calculates departure, generates message	Intelligence	LLM · AI SDK
Next.js	Web framework powering API routes (`/chats /messages / calendar`) and assistant UI	Experience	Full-stack Framework

Idea

Impact & Use cases

Priya, a marketing manager in Bengaluru, has back-to-back client meetings across the city (Indiranagar → Koramangala → Whitefield). She's constantly stressed about being late due to unpredictable traffic and metro delays.

Result:

- AI Commute Assistant reads her Google Calendar and detects her 3 PM meeting in Whitefield.
- Checks TomTom Traffic API: Heavy congestion on ORR; metro Line 1 is delayed.
- AI SDK calculates: "Leave by 2:00 PM. Take BMTC Bus 400A to Whitefield (ETA: 2:45 PM). Metro is delayed by 15 mins."
- Telegram bot sends the reminder at 1:30 PM with the updated route.

Rahul, a freelance designer, works from co-working spaces across Bengaluru. He often loses track of time and misses his own deadlines for client calls due to poor time estimation.

Result:

- AI Commute Assistant syncs with his Google Calendar and sees a client call at 11 AM in HSR Layout.
- TomTom Traffic API shows light traffic but no direct bus routes.
- AI SDK suggests: "Leave by 10:15 AM. Book an Ola Auto (ETA: 10:50 AM). Cost: ₹120."
- Telegram bot sends the reminder at 9:45 AM with the auto booking link.

Impact & Use cases

Ananya, a college student in Bengaluru, relies on BMTC buses and walking to attend classes and part-time jobs. She often misses buses due to unclear schedules and ends up late.

Result:

- **AI Commute Assistant pulls her class schedule from Google Calendar (9 AM lecture in Malleshwaram).**
- **TomTom Traffic API shows moderate traffic; Namma Metro is on time.**
- **AI SDK advises: "Leave by 7:45 AM. Take Metro Line 2 to Malleshwaram (ETA: 8:30 AM). Bus 244A is delayed."**
- **Telegram bot sends the reminder at 7:15 AM with metro platform info.**

Why this Scales

- Urbanization: 40%+ of India’s population will live in cities by 2030 (World Bank).
- Traffic Chaos: Bengaluru, Mumbai, and Delhi rank among the top 10 most congested cities globally (TomTom Index).
- Smartphone Penetration: 800M+ Indians use smartphones (Statista), enabling Telegram bot adoption.
- API Ecosystem: Google Calendar, TomTom, and Telegram APIs are globally available, making expansion seamless.
- Behavioral Shift: Users increasingly expect proactive AI (e.g., 60% of millennials use AI assistants daily).

Idea

Differentiation

Why this solution stands out is that it does not wait for the user to remember their schedule. It turns a routine problem into an automated, context-aware system that watches Google Calendar, checks TomTom traffic, and sends the reminder through Telegram at the right moment. That makes the experience practical, low-friction, and immediately useful in daily life, especially for users who commute, travel often, or manage packed calendars.

What makes it stronger than a basic reminder app is the intelligence layer. The AI SDK is not just generating text, it is helping decide when a reminder should happen and how it should be phrased. By combining calendar events, live traffic, and conversational delivery, the product adapts to real-world conditions instead of relying on fixed alarms. That creates a smarter reminder engine that feels personalized without requiring constant user input.

The biggest advantage is the end-to-end loop: one user action, the start command in Telegram, unlocks an automated workflow that keeps working in the background. This means the product is not only useful, but also sticky. Users get value before they even ask for it. That kind of proactive behavior builds trust, saves time, and creates a distinct experience that is harder for simple notification tools to replicate.

Strong moat:

1. Data-driven context moat: the product combines calendar, traffic, and AI reasoning into a single reminder workflow, which is more difficult to copy than a standalone bot or reminder app.
2. Workflow moat: once users connect their calendar and start using Telegram reminders, the system becomes part of their daily routine, creating switching costs and long-term retention.

A business "moat," or economic moat, is a sustainable competitive advantage that protects a company's long-term profits and market share from competitors, a concept popularized by Warren Buffett. Similar to a castle's moat, it creates a defensive barrier, allowing companies to maintain high margins and outperform rivals

Idea

Demo Link

GitHub Repo: https://github.com/AditiShastri/path_pilot
Demo Link : https://drive.google.com/file/d/1v168hykvaCE5abebv18Sy7TXcvavadiB/view?usp=sharing
Document submitted: Yes

Idea

<Brownie Slide>

Further Chat related output

Execution Trace

Planning commute

Checking mock events, routes, travel modes, conflicts, and bookings.

Status: Completed

Okay, I've planned your commute to Electronic City from Whitefield.

It looks like you have a **Client Presentation at Electronic City Bangalore today at 2:30 PM**. To make it on time, you should **leave by 1:22 PM**.

The recommended mode of travel is by **cab**, which will take approximately **44 minutes**. The estimated cost is **Rs. 680**. According to TomTom, there is currently no traffic delay, so your travel time should be accurate.

You'll receive a Telegram message with these details 10 minutes before your departure time.

Chat Usage

Token usage for this conversation is stored temporarily in memory.

Prompt	Cache	Reply	Total
14220	5462	1002	20684

Turn	Role	Prompt	Cached	Reply	Total
1	Assistant	2945	0	116	3061
4	Assistant	2997	0	223	3220
6	Assistant	3689	2697	162	6548
8	Assistant	4589	2765	501	7855